Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

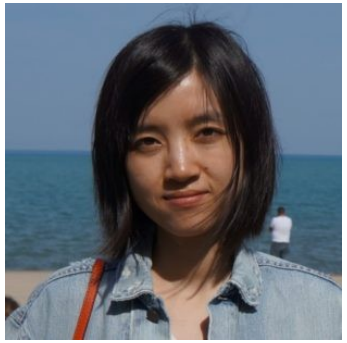
Speakers



Ricardo Aravena

AI Infrastructure Lead · Nunchux AI

CNCF TOC Alternate (GB)



Rainie Li

Pinterest

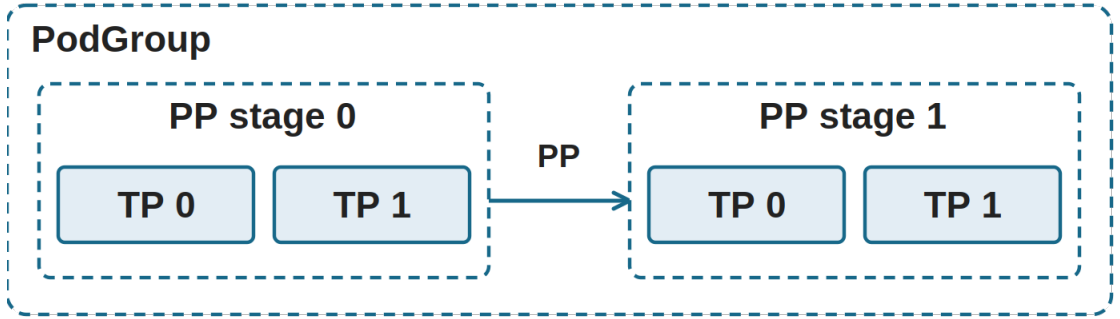
Apache YuniKorn PMC member

01

Unified scheduling

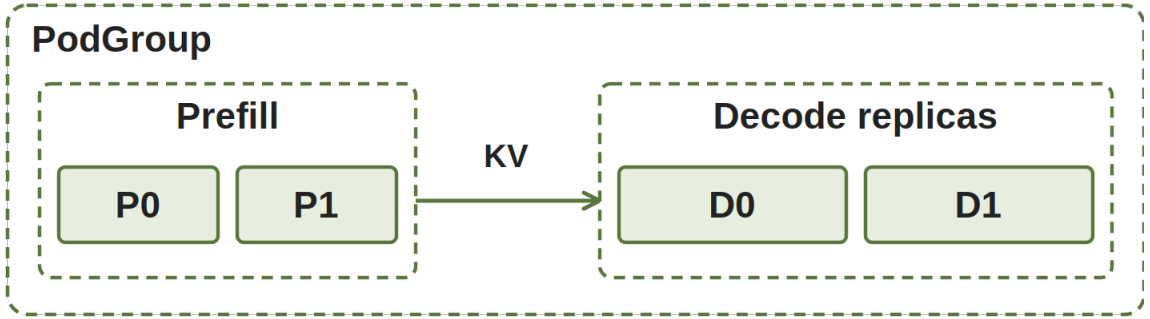
Different workloads, shared infrastructure

Training



TP / PP gangs

Inference



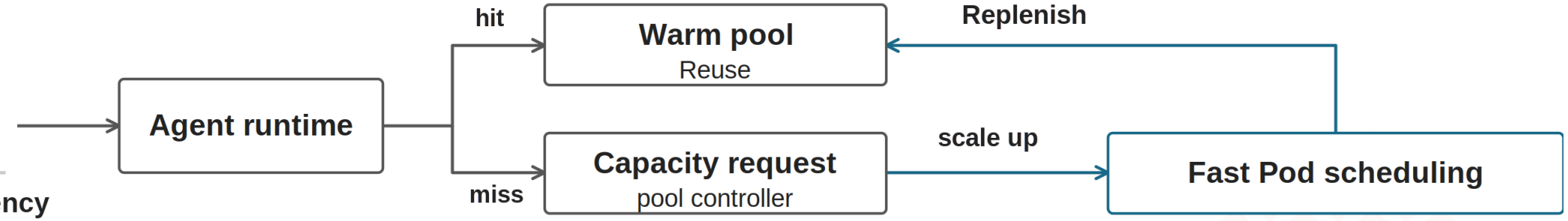
Role groups and network locality

Agents

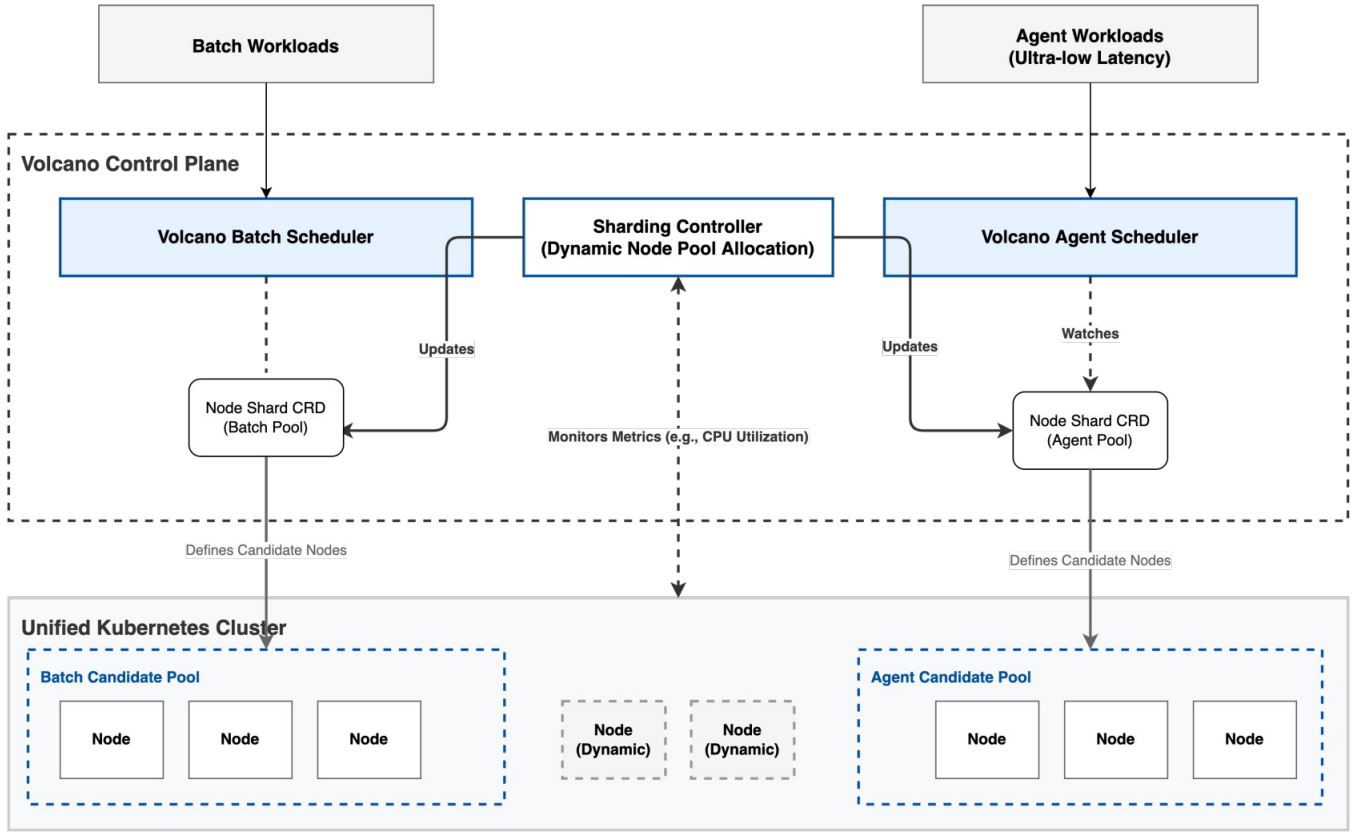
Bursty arrivals



Low startup latency



Unified scheduling

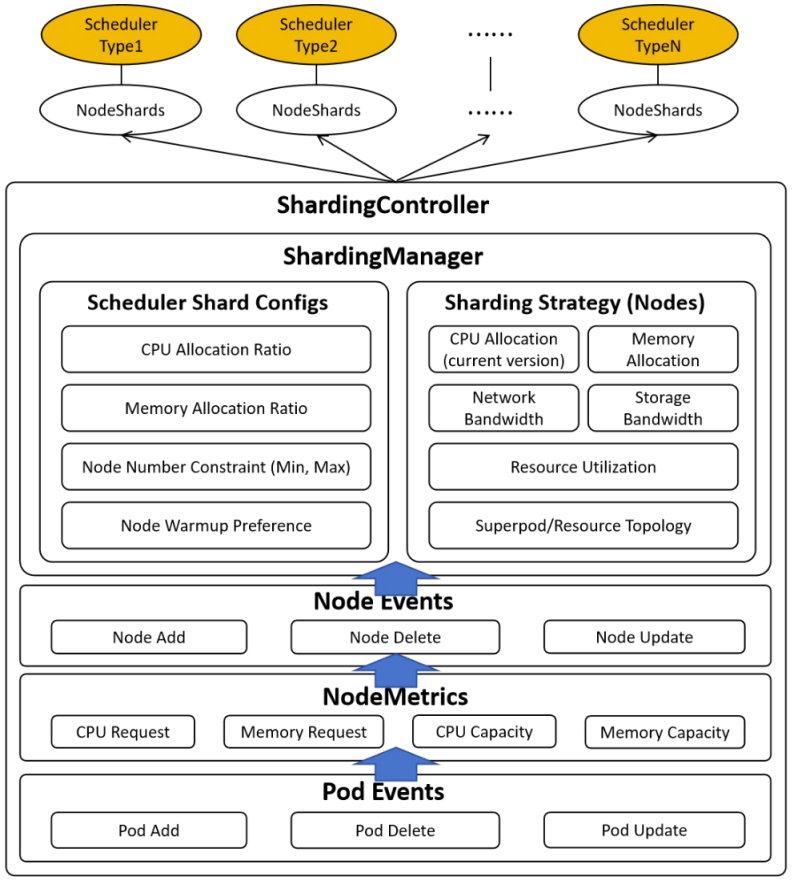


Batch and fast paths

Binder conflict checks

NodeShard coordination

NodeShard coordination



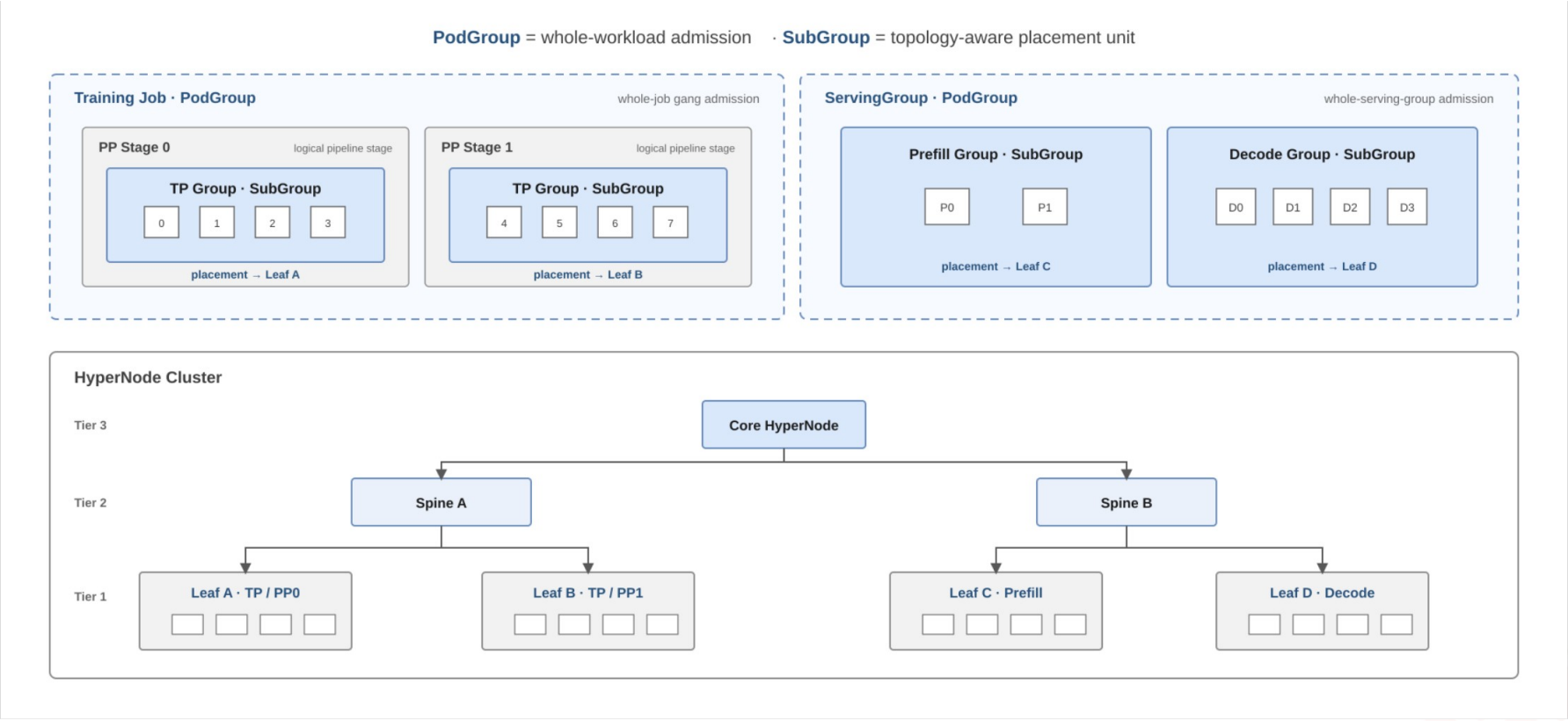
NodeShard

```
spec:
  nodesDesired:
  - node1
  - node2
status:
  nodesInUse:
  - node0
  - node1
  nodesToRemove:
  - node0
  nodesToAdd:
  - node2
```

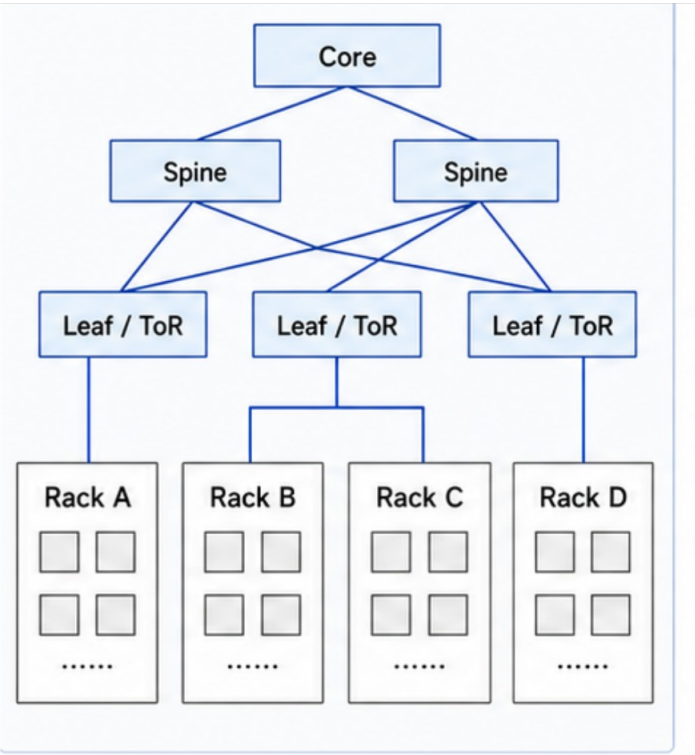
02

Multi-level gang & network topology

PodGroup and SubGroup



HyperNode



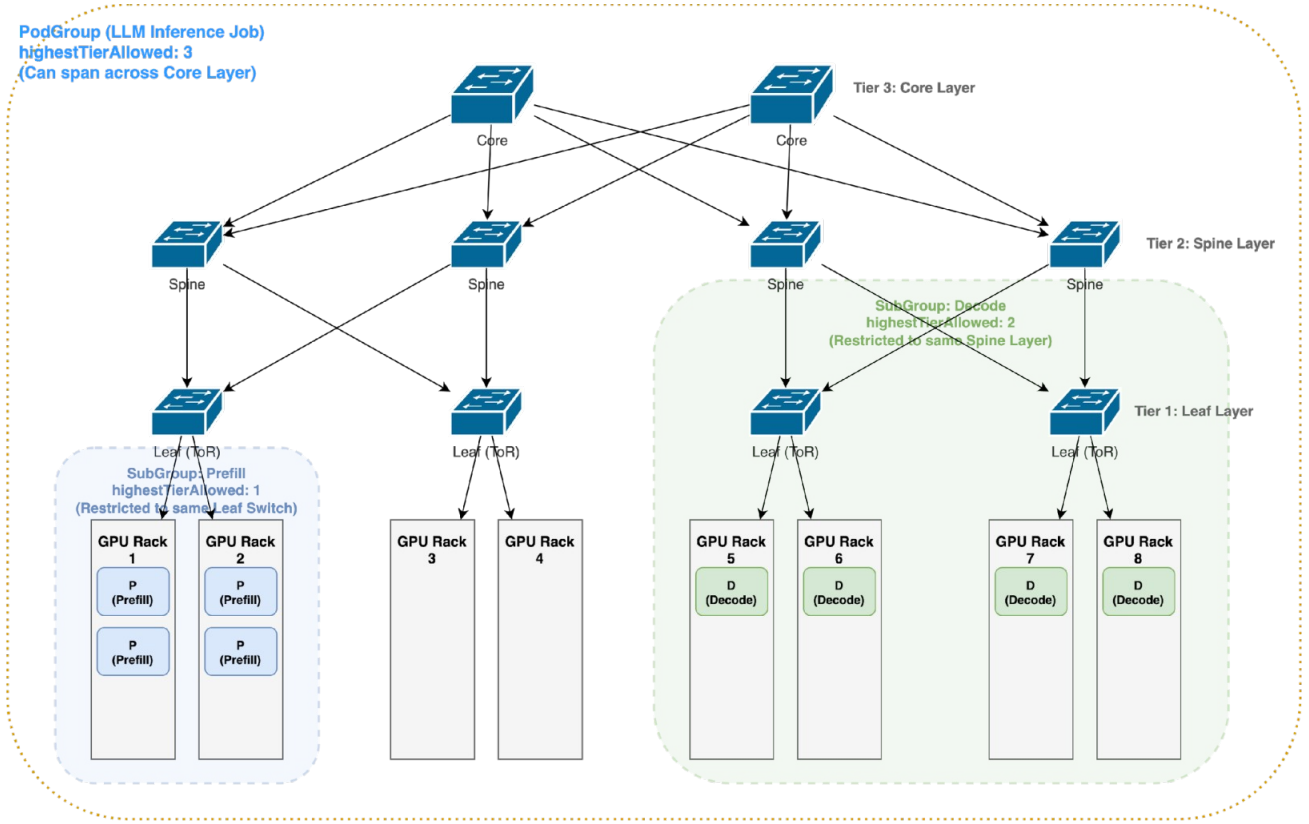
Parent HyperNode

```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

Leaf HyperNode

```
tier: 1
tierName: leaf
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

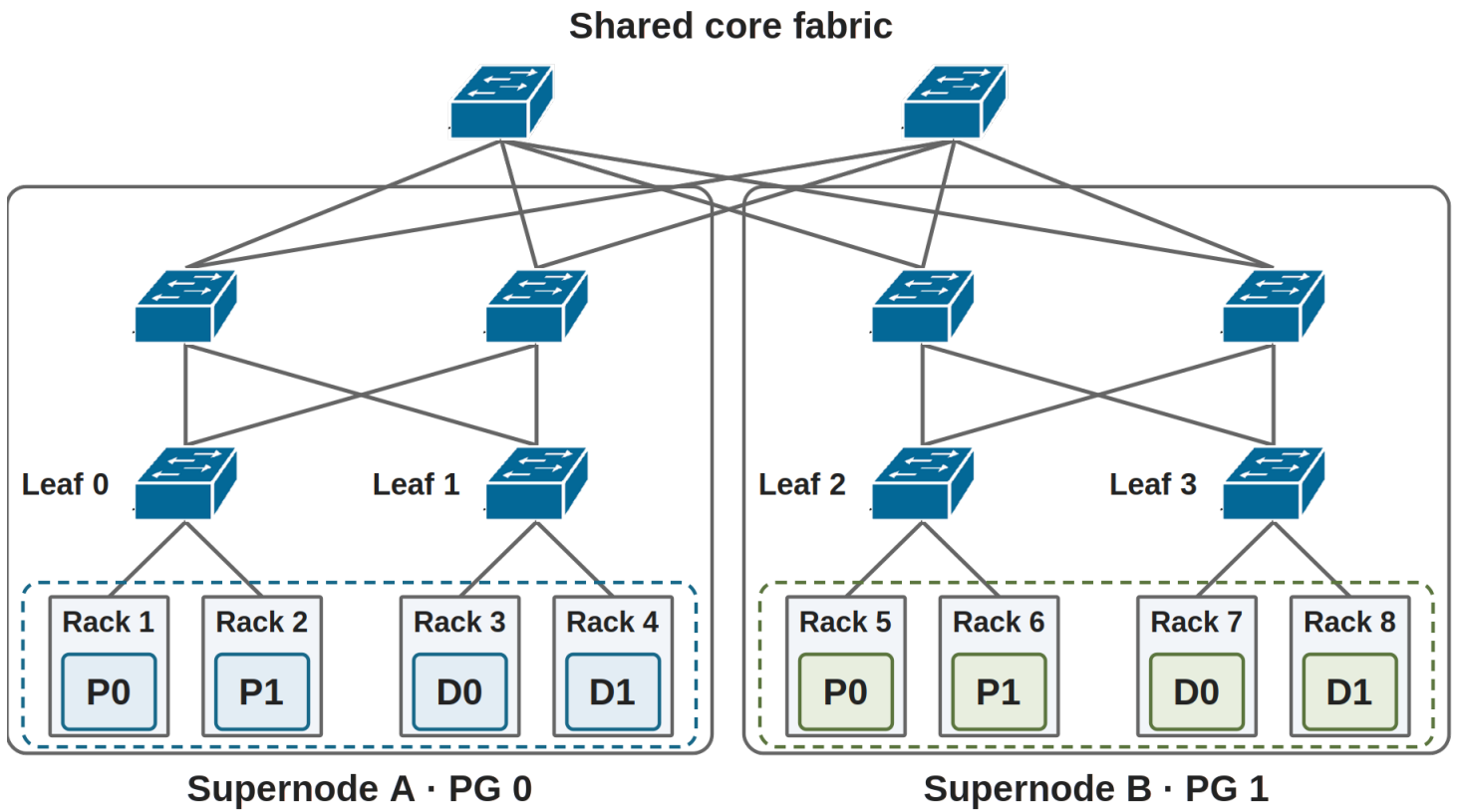
Place groups at different topology levels



PodGroup.spec

```
minMember: 8
networkTopology:
  mode: hard
  highestTierName: core
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierName: leaf
- name: decode
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierName: spine
```

PodGroup anti-affinity



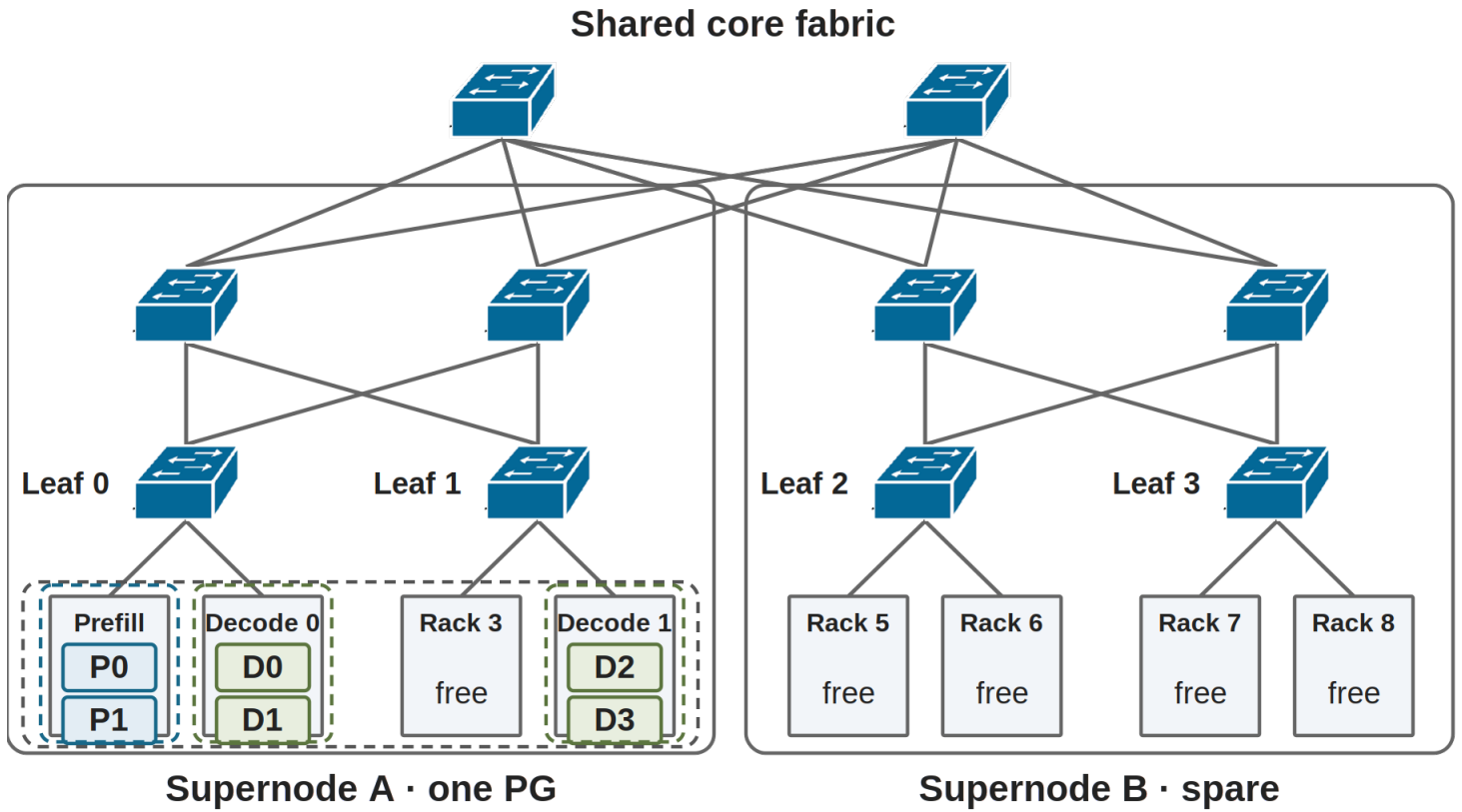
PodGroup.spec

```
networkTopology:
  mode: hard
  highestTierName: supernode
topologyAffinity:
  podGroupAntiAffinity:
    required:
    - podGroupSelector:
        matchLabels:
          app: model-a
      topologyTierName: supernode
```

Each PodGroup: app=model-a

Kthena integration proposed

SubGroup affinity and anti-affinity



Proposed

Affinity at supernode level

Prefill and Decode

Anti-affinity at leaf level

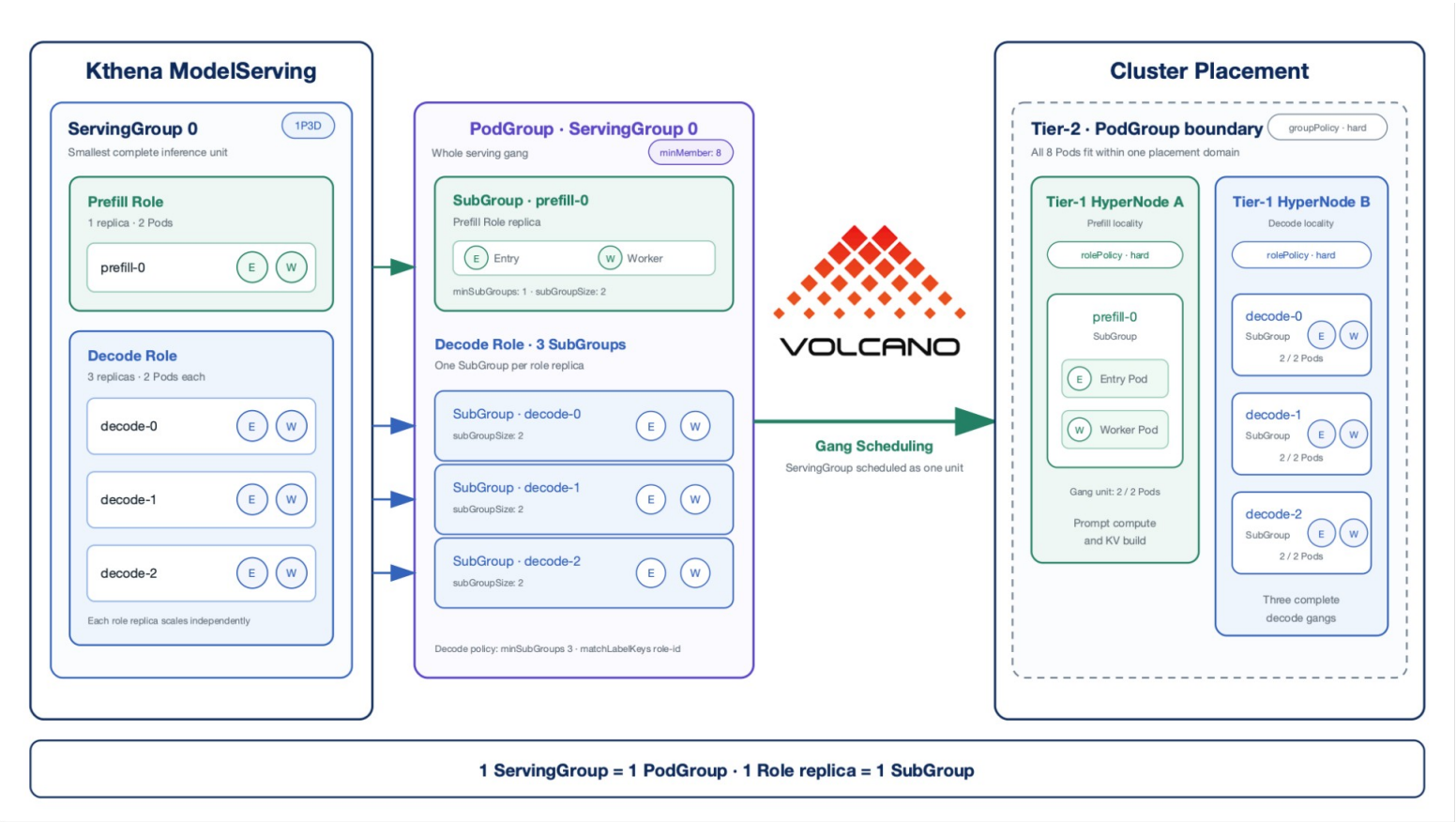
Decode replicas

VOLCANO

03

Ecosystem

Kthena



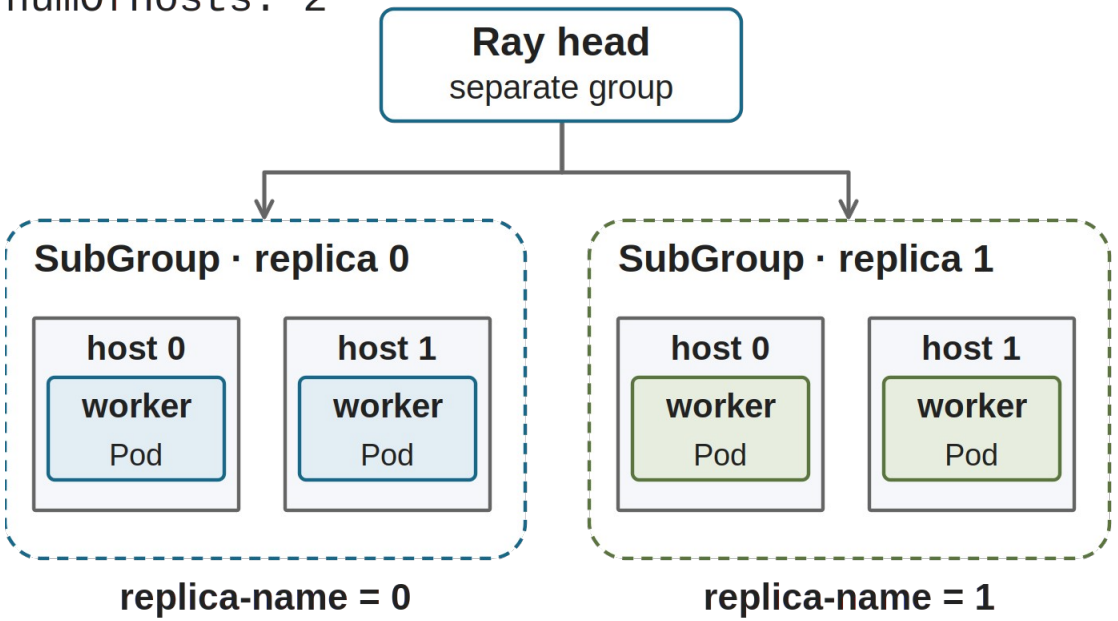
ModelServing.spec

```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  roles:
  - name: prefill
    replicas: 1
    workerReplicas: 1
  - name: decode
    replicas: 3
    workerReplicas: 1
```

KubeRay

Ray worker group

```
minReplicas: 2
numOfHosts: 2
```



PodGroup.spec

```
subGroupPolicy:
- name: worker
  labelSelector:
    matchLabels:
      ray.io/group: worker
  matchLabelKeys:
  - ray.io/worker-group-replica-name
  minSubGroups: 2
  subGroupSize: 2
```

Topology support in progress

Grove



Grove PodGang

PodGang.spec.podGroups

prefill `minReplicas: 2`

Pod 0 | Pod 1

decode `minReplicas: 4`

Pod 0 | Pod 1 | Pod 2 | Pod 3

6 Pods

one policy

one policy

Volcano PodGroup

PodGroup.spec.subGroupPolicy

prefill `subGroupSize: 2`

`minSubGroups: 1`

decode `subGroupSize: 4`

`minSubGroups: 1`

`minMember: 6`

Topology constraints not supported yet

Next integrations



RBG

In progress

RoleInstance → SubGroup



Kubeflow

Planned

Training groups and topology



LWS

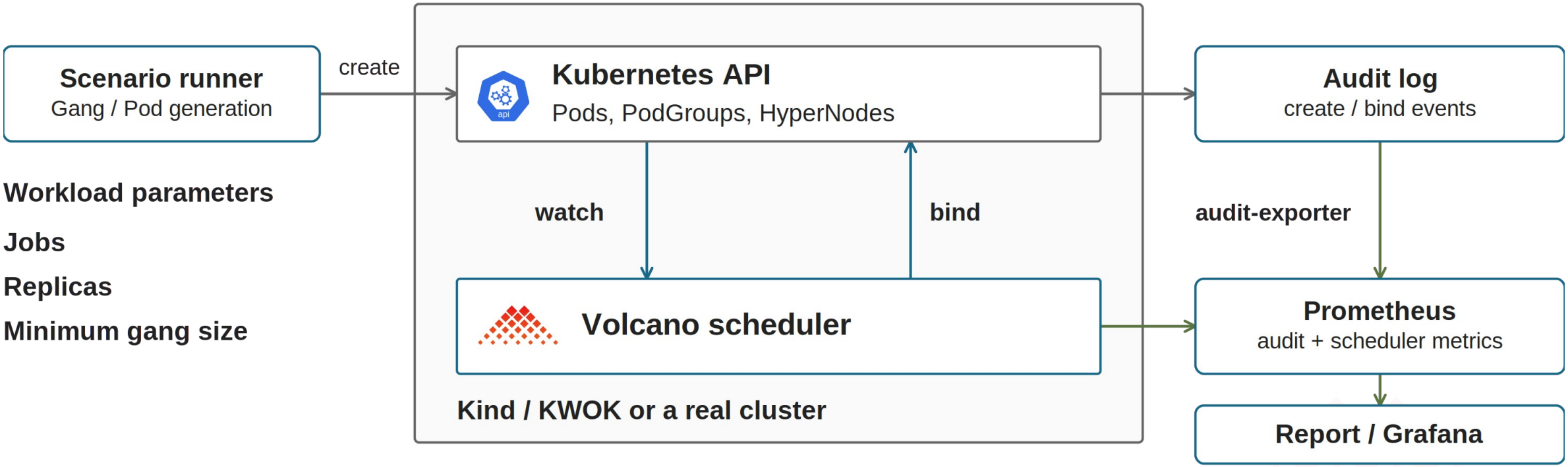
Planned

Multi-group support

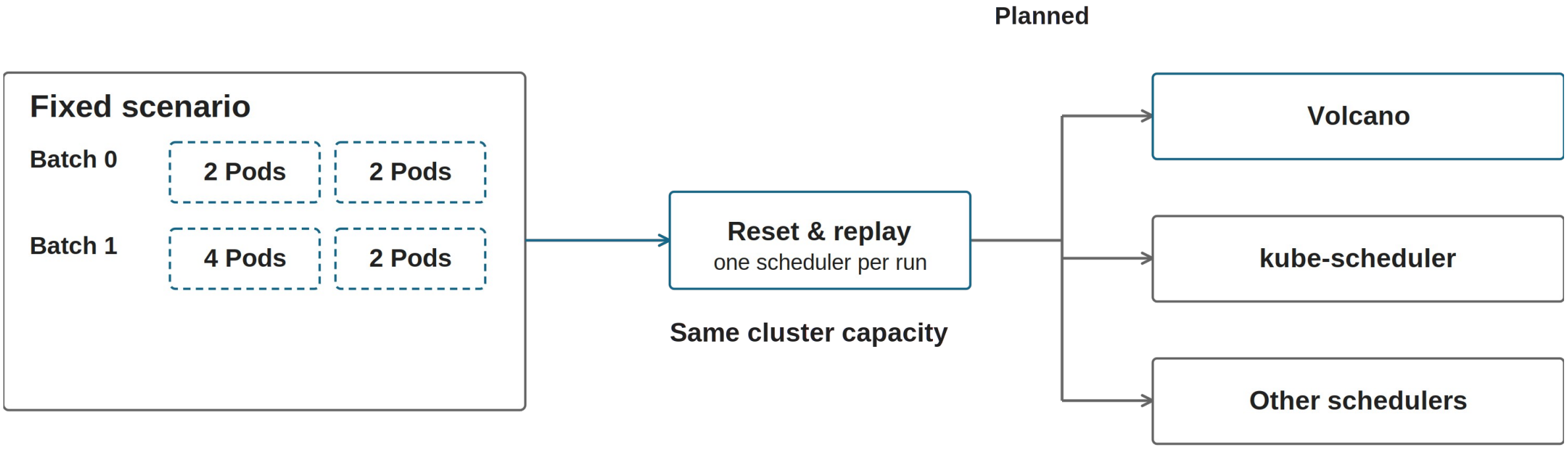
04

Benchmark

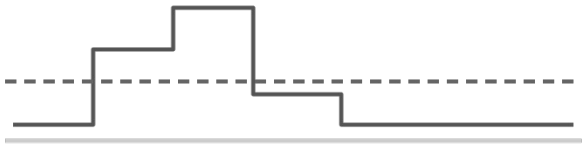
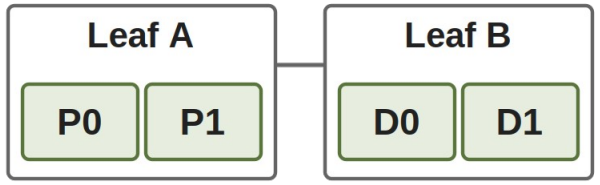
Benchmark framework



Comparing schedulers



Benchmark scenarios

Scenario		Vary	Measure
Bursts		**Arrival rate**	**Tail scheduling latency**
Gang size		**Group size and fragmentation**	**Complete gangs placed**
Topology		**Topology constraints**	**Placement correctness**

What’s next

- SubGroup affinity and anti-affinity
- RBG, Kubeflow and LWS integration
- Cross-scheduler and topology benchmarks

Volcano community



Thank you

volcano.sh

GitHub

Slack

Questions?